

EX. NO: 2**STACK AND APPLICATION****DATE:****QUESTION 1:**

2 (a) Implement Stack operations using arrays.

AIM:

To implement the basic Stack operations using arrays in C, such as Push, Pop, and Display.

ALGORITHM:

1. Start the program.
2. Declare an array to represent the stack and initialize top = -1.
3. Display the stack menu with the following operations:
 - Push
 - Pop
 - Display
 - Exit
4. Read the user's choice.
5. If the choice is Push:
 - Check if the stack is full.
 - If full, display "Stack Overflow".
 - Otherwise, increment top and insert the element into the stack.
6. If the choice is Pop:
 - Check if the stack is empty.
 - If empty, display "Stack Underflow".
 - Otherwise, remove the top element and decrement top.
7. If the choice is Display:
 - Check if the stack is empty.
 - If empty, display "Stack is Empty".
 - Otherwise, display all the elements from top to 0.
8. Repeat Steps 4–7 until the user chooses Exit.
9. Stop the program.

PROGRAM:

```
#include <stdio.h>
#define SIZE 100

int stack[SIZE];
int top = -1;

void push()
{
    int item;

    if (top == SIZE - 1)
    {
        printf("Stack Overflow\n");
    }
    else
    {
        printf("Enter element: ");
        scanf("%d", &item);

        top++;
        stack[top] = item;

        printf("Element inserted\n");
    }
}

void pop()
{
    if (top == -1)
    {
        printf("Stack Underflow\n");
    }
    else
    {
        printf("Deleted element = %d\n", stack[top]);
        top--;
    }
}

void peek()
{
    if (top == -1)
    {
        printf("Stack is Empty\n");
    }
    else
    {
        printf("Top element = %d\n", stack[top]);
    }
}
```

```
void display()
{
    int i;

    if (top == -1)
    {
        printf("Stack is Empty\n");
    }
    else
    {
        printf("Stack elements are:\n");

        for (i = top; i >= 0; i--)
        {
            printf("%d ", stack[i]);
        }

        printf("\n");
    }
}

int main()
{
    int choice;

    while (1)
    {
        printf("\n----- STACK MENU ----- \n");
        printf("1. Push\n");
        printf("2. Pop\n");
        printf("3. Peek\n");
        printf("4. Display\n");
        printf("5. Exit\n");

        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                push();
                break;

            case 2:
                pop();
                break;

            case 3:
                peek();
                break;

            case 4:
                display();
                break;
```

```

case 5:
    return 0;

default:
    printf("Invalid Choice\n");
}
}

```

OUTPUT:

```

[ita2536@mritlin harshan]$ vi ex2.1.c
[ita2536@mritlin harshan]$ cc ex2.1.c
[ita2536@mritlin harshan]$ ./a.out
----- STACK MENU -----
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element: 2
Element inserted

----- STACK MENU -----
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element: 4
Element inserted

----- STACK MENU -----
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element: 6
Element inserted

----- STACK MENU -----
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 4
Stack elements are:
6 4 2

----- STACK MENU -----
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 5
[ita2536@mritlin harshan]$

```

RESULT:

Thus, the C program to implement Stack operations using arrays was implemented and executed successfully.

QUESTION:

2 (b) Implement Infix to Postfix conversion using Stack.

AIM:

To implement the Infix to Postfix conversion using the Stack data structure in c.

ALGORITHM:

1. Start the program.
2. Read the infix expression from the user.
3. Initialize an empty stack for storing operators.
4. Scan the infix expression from left to right.
5. If the symbol is an operand, add it to the postfix expression.
6. If the symbol is '(', push it onto the stack.
7. If the symbol is ')', pop and add operators to the postfix expression until '(' is encountered. Remove the '(' from the stack.
8. If the symbol is an operator, pop operators with higher or equal precedence from the stack and add them to the postfix expression. Then push the current operator onto the stack.
9. After scanning the entire expression, pop all the remaining operators from the stack and add them to the postfix expression.
10. Display the postfix expression.
11. Stop the program.

PROGRAM:

```
#include <stdio.h>
#include <ctype.h>
```

```
char stack[20];
int top = -1;
```

```
void push(char x)
{
    stack[++top] = x;
}
```

```
char pop()
{
    return stack[top--];
}
```

```
int priority(char x)
{
    if(x == '+' || x == '-') return 1;
    if(x == '*' || x == '/') return 2;
    return 0;
}
```

```

int main()
{
    char infix[20], postfix[20];
    int i = 0, j = 0;

    printf("Enter Infix: ");
    scanf("%s", infix);

    while(infix[i] != '\0')
    {
        if(isalnum(infix[i]))
            postfix[j++] = infix[i];
        else
        {
            while(top != -1 && priority(stack[top]) >= priority(infix[i]))
                postfix[j++] = pop();

            push(infix[i]);
        }
        i++;
    }

    while(top != -1)
        postfix[j++] = pop();

    postfix[j] = '\0';

    printf("Postfix = %s", postfix);

    return 0;
}

```

OUTPUT:

```

[ita2536@mritlin harshan]$ vi ex2.2.c
[ita2536@mritlin harshan]$ cc ex2.2.c
[ita2536@mritlin harshan]$ ./a.out
Enter Infix: a+b-c*h
Postfix = ab+ch*-
[ita2536@mritlin harshan]$ █

```

RESULT:

Thus, the C program to convert an Infix expression into a Postfix expression using Stack was executed successfully.